Securitatea sistemelor distribuite

Securitatea in sisteme distribuite

Securitatea – aspecte tratate in acest curs

- comunicatiei – intre utilizatori si/sau procese situate in calculatoare diferite
- accesului – un utilizator/proces utilizeaza doar serviciile/resursele la care are dreptul (autorizare)
- gestiunea securitatii (cheilor, utilizatorilor, grupurilor)

Sistemele distribuite sunt mai vulnerabile

- control descentralizat
- accesul se face de la distanta
- tot mai multe aplicatii critice (Internet banking) sunt distribuite

Securitatea

Probleme de securitate (ce inseamna securitatea?)

- Confidentialitate
- Integritate - acopera
  - integritatea datelor
  - integritatea originii (autentificarea)
- Disponibilitate

Tipuri de amenintari

Atac	Canal	Obiect
Inspectie	**Citire continut mesaj**	**Citire date continute de obiect**
Fabricare	**Inserare mesaj**	**Parodiere obiect**
Modificare	**Schimbare mesaj**	**Schimbare date incapsulate**
Intrerupere	**Prevenirea transferului**	**Denial of service**

Construire sistem sigur - etape

1. Specificarea politicii de securitate (cerinte)

Politica - Ce actiuni sunt permise unei entitati (utilizator, serviciu, masina, proces etc.)

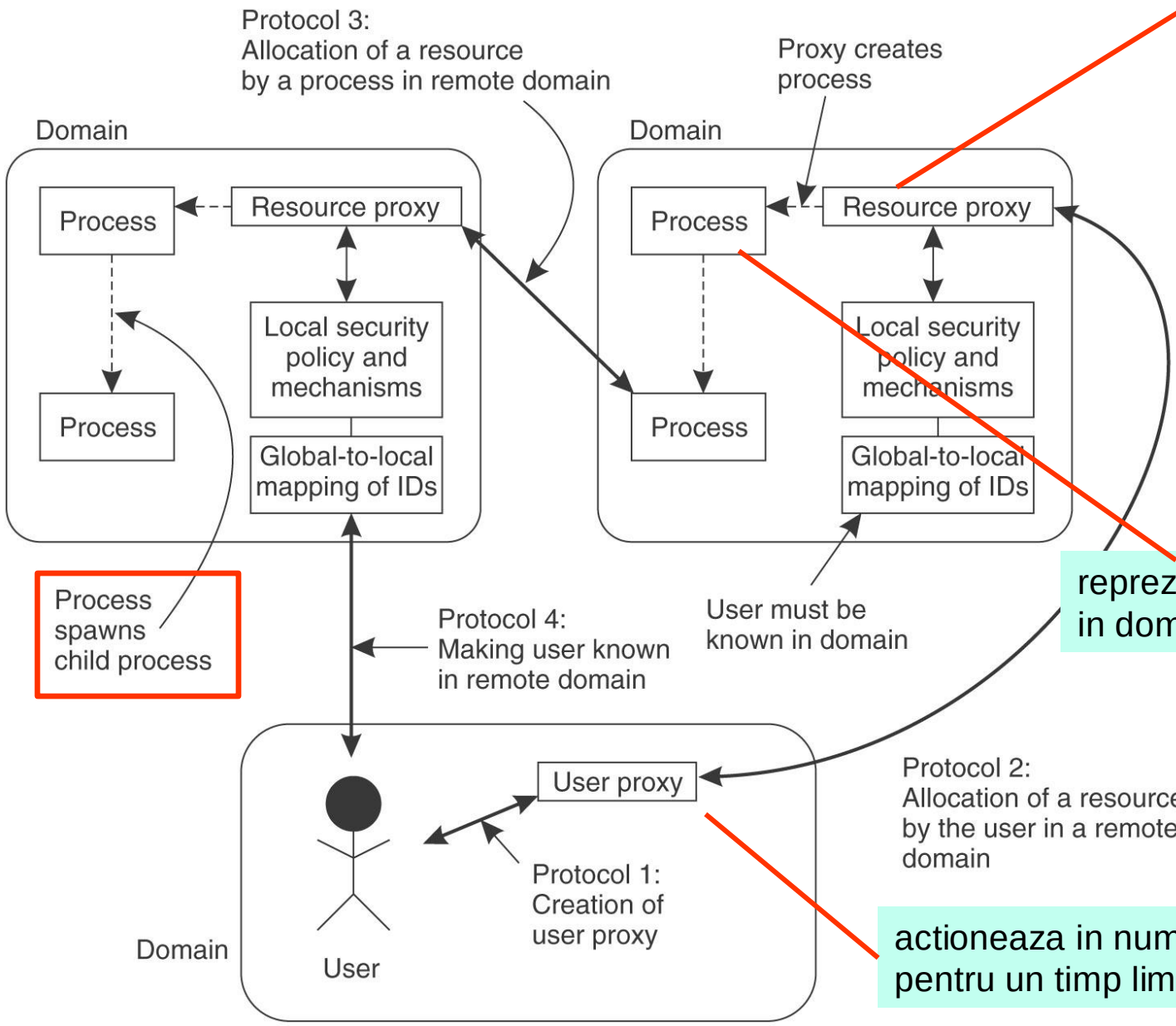
Exemplu: Globus

- Exista mai multe domenii administrative cu politici diferite
- Operatii locale – conform politicii locale
- Operatii globale - controlul accesului este subiectul securitatii locale
  - identificatorul initiatorului sa fie cunoscut local
    - autentificarea globala tine loc si de autentificare locala
    - cer autentificare mutuala
    - entitatile pot delega privilegii proceselor
- Pentru procese din acelasi domeniu si actionand in contul aceluiasi utilizator - credentialele pot fi partajate in grup

2. Alegerea mecanismelor de securitate

- Focus pe operatii globale
- **Ex.** Globus
  - Reprezentarea unui utilizator intr-un domeniu la distanta
  - Alocarea resurselor dintr-un domeniu la distanta, utiliz. sau proxy

3. Implementarea (entitati si protocoale)

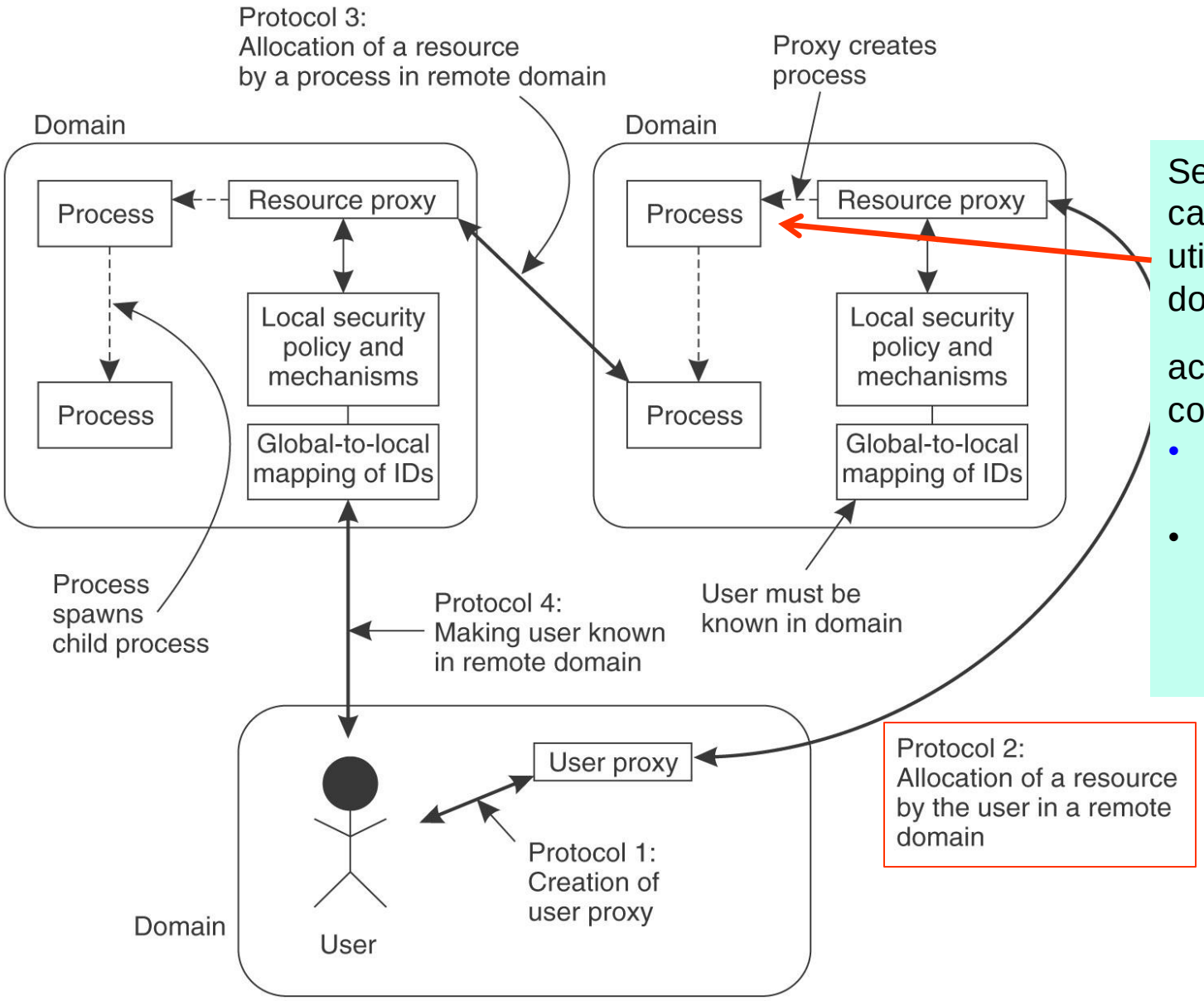


folosit de user proxy pentru a transforma op globale pe resurse in op locale compatibile cu politica de securitate

reprezinta utilizatorul in domeniul distant

actioneaza in numele user pentru un timp limitat

Protocolul 2 – alocare resurse



Se creaza un proces care reprezinta utilizatorul in domeniul distant;

acces la resursa conform

- deciziilor locale de acces
- si unor credentiale (Cp) capatate prin protocolul 2.

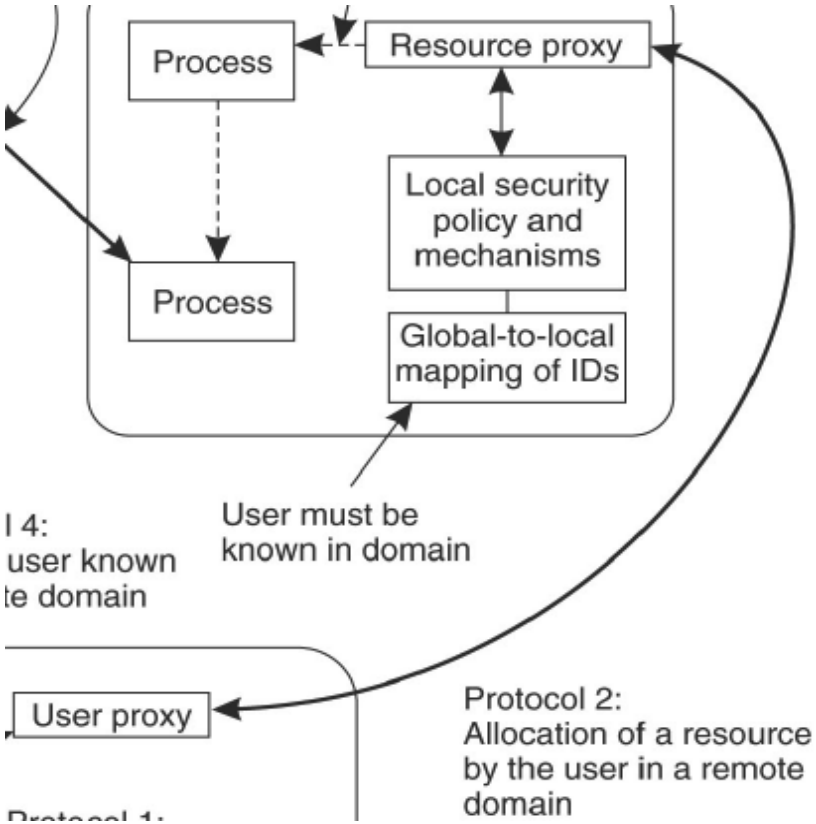
Protocolul 2 continuare

1. user proxy - UP si resource proxy - RP se autentifica reciproc folosind certificate Cup and Crp. RP verifica daca credentialele UP nu au expirat.

2. UP trimite RP o cerere de alocare semnata SigUP {specificatie alocare}.

3. RP verifica daca UP este autorizat local sa faca alocarea resursei.

4. Daca da, RP creaza un tuplu RESOURCE-CREDENTIALS continand numele utilizatorului, numele resursei etc.



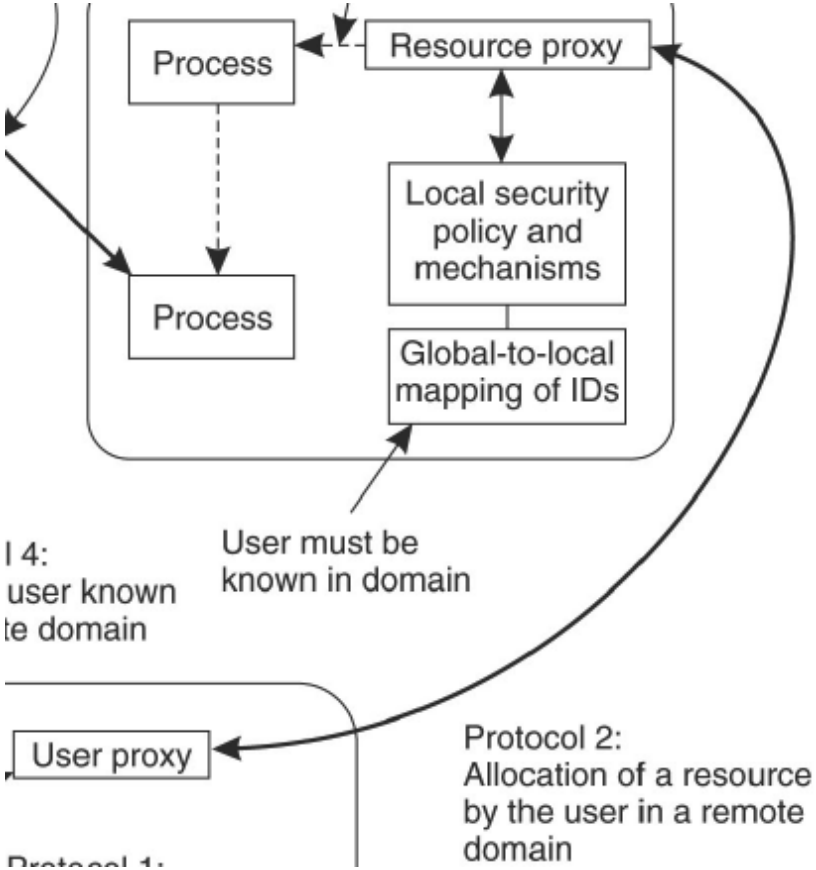
Protocolul 2 – continuare

5. RP paseaza securizat RESOURCE-CREDENTIALS lui UP.

6. UP examineaza RESOURCE-CREDENTIALS si, daca o aproba, semneaza tuplul producand Cp, credentialele pentru resursa ceruta.

7. UP paseaza securizat Cp lui RP.

8. RP aloca resursa si paseaza Cp noului proces. (acest transfer tine cont ca RP si procesul sunt in acelasi domeniu de incredere.)



Trusted Computer Base (TCB)

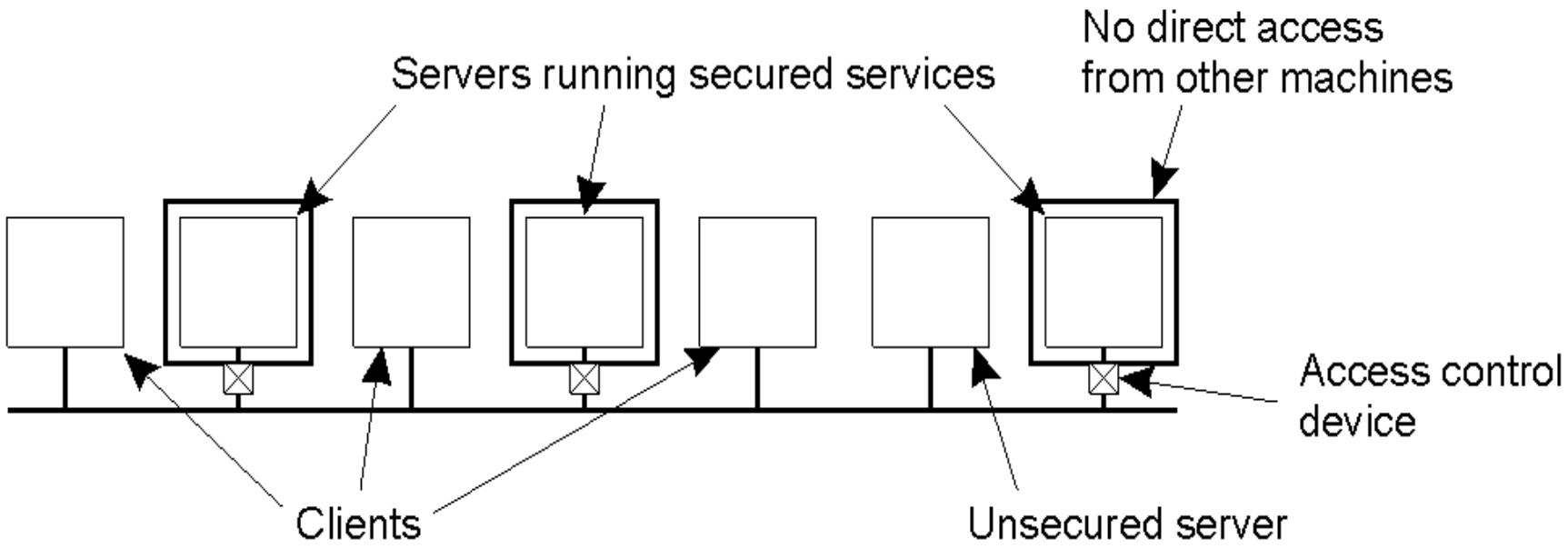
Totalitatea componentelor si mecanismelor software si hardware cerute pentru a implementa o politica de securitate

Principiul RISSC (Reduced Interfaces for Secure System Components) aplicat pentru securizarea sistemelor distribuite.

- orice server cu cerinte critice de securitate este plasat pe o masina separata, izolat de sistemele utilizatorului
- clientii si aplicatiile pot accesa serverul doar prin interfete de retea sigure.

Solutie similara: Containerele - cresc gradul de securitate prin izolare

- ex. SELinux si CoreOS; Docker



4. *Asigurare* ca mecanismele functioneaza corect

Metode formale

- aplicare in toate etapele: specificare, proiectare, implementare

Increderea (trust) - convingerea ca entitatea realizeaza politica de securitate pentru care a fost conceputa

- caracteristica generala legata de perceptia subiectului: exista mai multe grade de incredere
- prin contrast, securitatea este caracteristica binara a obiectului, care poate fi sigur sau nu

Pentru credibilitate, evaluarea (assessment) produselor se face

- in Conformitate cu standardele (ex. ITSEC – Information Technology Security Evaluation Criteria)
- de catre experti neutri

Criterii de evaluare

TCSEC - Trusted Computer System Evaluation Criteria

- cunoscut ca Orange Book (USA), standard DoD

ITSEC - Information Technology Security Evaluation Criteria

- publicata in 1990 de Franta, Germania, Olanda si UK

Common Criteria - Common Criteria for Information Technology Security Evaluation

- standard ISO/IEC 15408 din 2005
- derivat din ITSEC, TCSEC, CTCPEC (standard Canadian)
- revizuit in 2009 (ISO/IEC 15408-1:2009) si confirmat in 2015

Common Criteria

Clasifica produsele IT dupa nivelul de incredere (**Evaluation Assurance Level,** EAL1 - EAL7)

- EAL1: Functionally Tested
- EAL2: Structurally Tested
- EAL3: Methodically Tested and Checked
- EAL4: Methodically Designed, Tested, and Reviewed
- EAL5: Semiformally Designed and Tested
- EAL6: Semiformally Verified Design and Tested
- EAL7: Formally Verified Design and Tested

vezi: http://en.wikipedia.org/wiki/Evaluation_Assurance_Level

Autentificarea

Protocoale analizate

RADIUS - Remote Authentication Dial In User Service

- Protocol de Autentificare, Autorizare si Accounting pentru accesul la retele de pe echipamente mobile (Network Access si IP Mobility)

SECURE RPC

- Protocol pentru Autentificare si Confidentialitate in apelurile de proceduri la distanta

KERBEROS

- Protocol pentru Autentificare, Autorizare si Confidentialitate intr-un sistem distribuit

RADIUS - Remote Authentication Dial In User Service

Standard "de facto" pentru autentificarea, autorizare si contabilizare (AAA – Authentication, Authorization, Accounting) pentru utilizatorii care se conecteaza la- si folosesc servicii de retea

Ulterior, standard IETF (Internet Engineering Task Force)

Folosit de ISP-uri si intreprinderi pentru gestiunea accesului la Internet si retele private, retele wireless, servicii e-mail integrate etc.

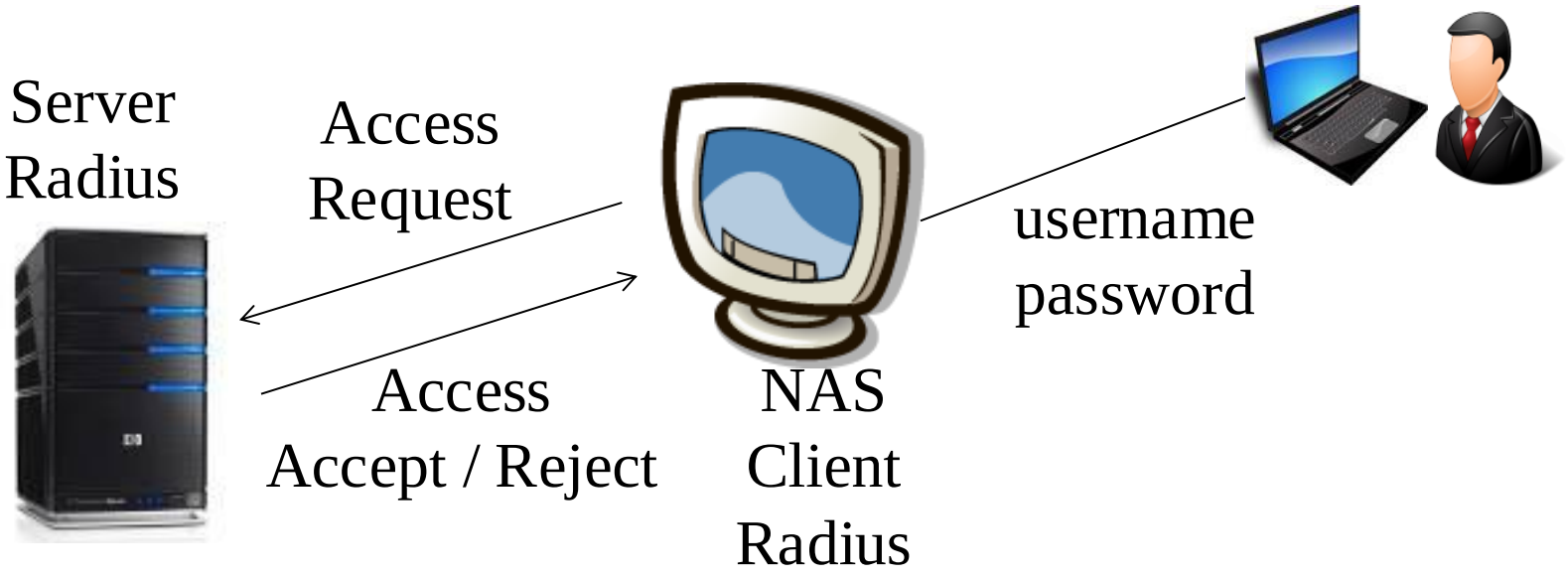
Folosit la distanta pe legaturi telco (de ex pentru mobilitate IP).

Contabilizeaza resursele utilizate intr-o sesiune (timp, pachete …)

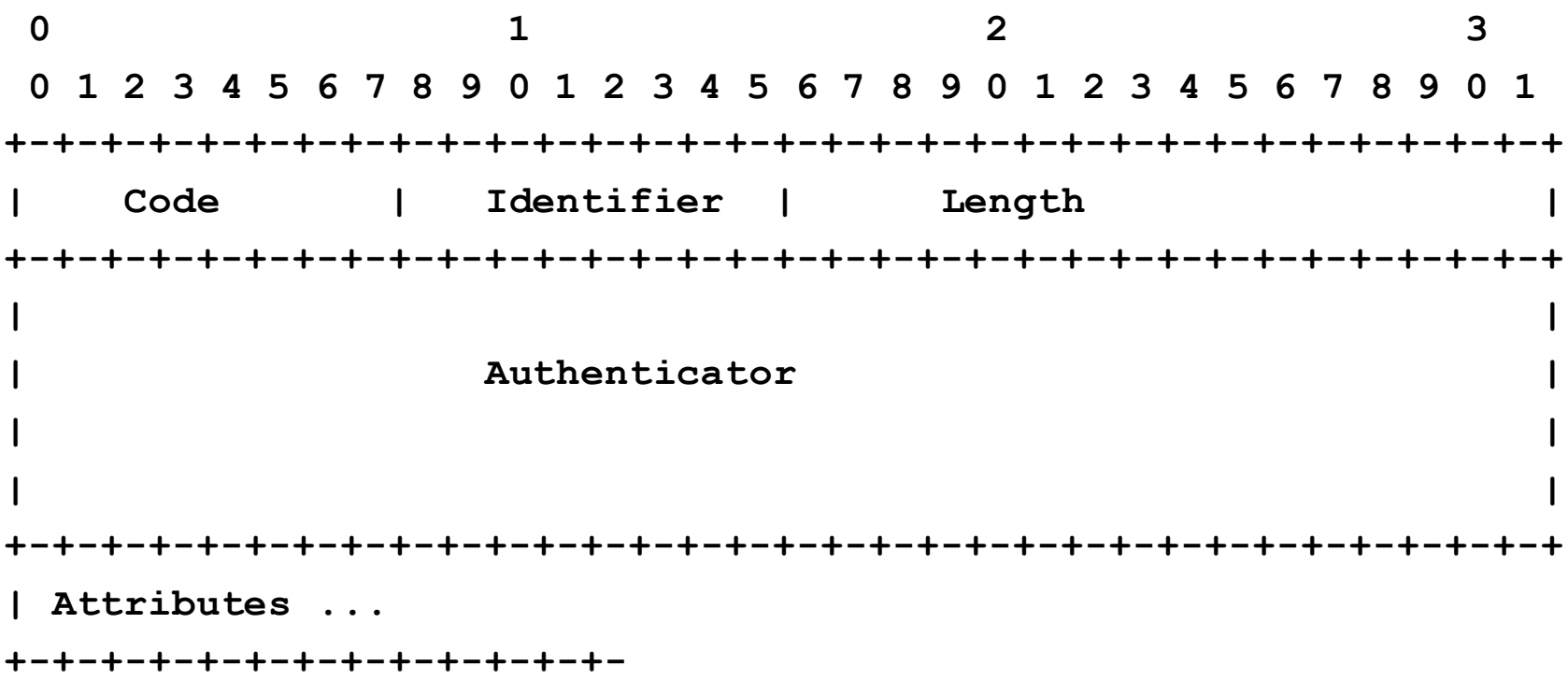
RADIUS

Protocol

1. Utilizatorul intra in retea prin modem.
2. RADIUS client (NAS – Network Access Server) cere username si password.
3. RADIUS client trimite Access Request (codificat MD5).
4. RADIUS server autentifica utilizatorul si trimite Access Accept sau Access Reject la RADIUS client.



Pachetul RADIUS



Code	Description	Code	Description
1	Access-Request	2	Access-Accept
3	Access-Reject	4	Accounting-Request
5	Accounting-Response	11	Access-Challenge
12	Status-Server (experimental)	13	Status-Client (experimental)
255	Reserved		

Actiuni client

Creaza pachet Access-Request care include cel putin atributele User-Name si User-Password.

Identifier–ul cererii este generat de client

Pachetul Access-Request contine Request Authenticator, RA (16 octeti alesi aleator).

Doar User-Password protejat:

- Client si server partajeaza un secret S.
- Password impartit in blocuri de 16-octeti p1, p2, ... pn (cu ultimul bloc completat la 16 octeti)
- Blocurile de text cifrat sunt obtinute astfel:

  c1 = p1 XOR MD5(S + RA)
  c2 = p2 XOR MD5(S + c1)
  .
  .
  cn = pn XOR MD5(S + cn-1)

Atributul User-Password contine **c1+c2+...+cn**

(+ denota concatenarea).

Actiuni server plus client

Server

Primeste pachet RADIUS Access-Request.

Verifica daca are secret partajat cu clientul (daca nu, ignora).

Obtine password neprotejat (procedura similara client).

Foloseste BD autentificare pentru validare username si password.

- password valid => creaza pachet Access-Accept
- password invalid => creaza Access-Reject.
- Adauga Response Authenticator in camp Authenticator.

ResponseAuth =
MD5(Code+ID+Length+RequestAuth+Attributes+Secret)

(+ denota concatenarea).

Client

Gaseste cererea corespunzatoare raspunsului (cf. Identifier)

Calculeaza ResponseAuth (repetand operatii server) si compara rezultat cu camp din raspuns) – ignora daca nu corespund.

Secure RPC

Dezvoltat de Sun pentru sistemul de operare.

Bazat pe o combinatie de chei publice si chei secrete

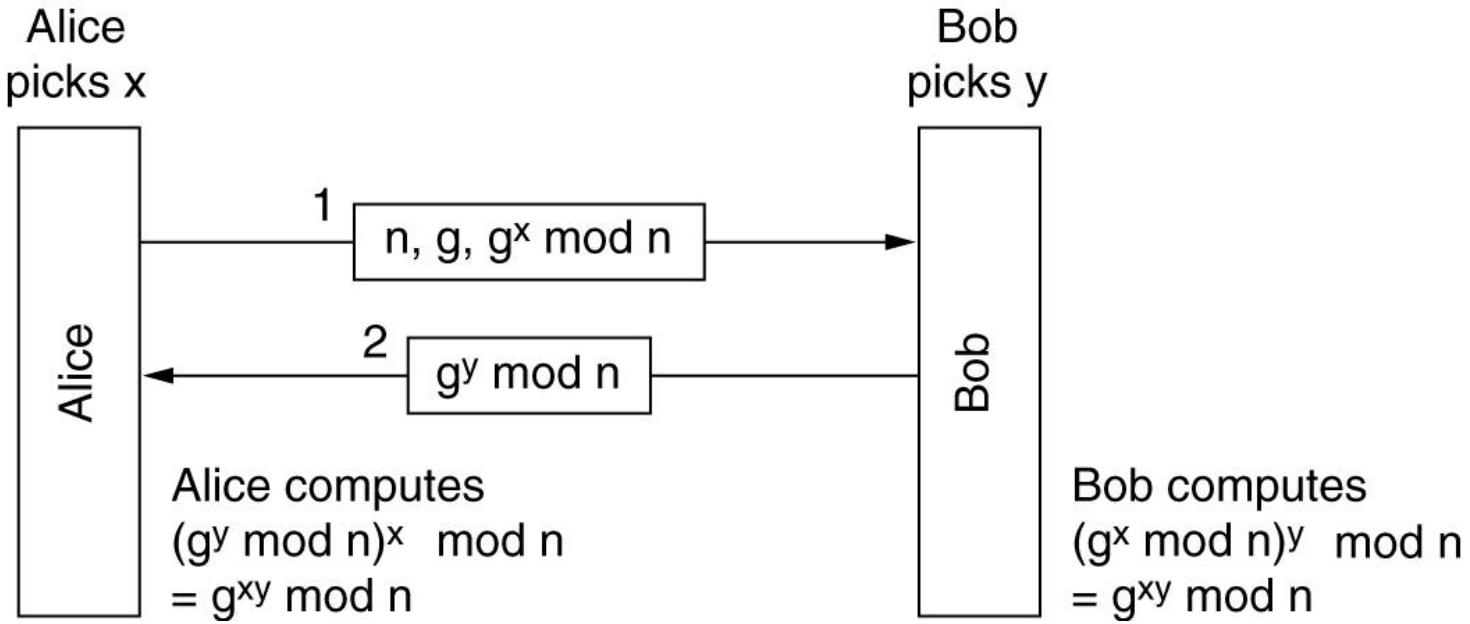
- DES pentru criptarea datelor
- Diffie-Hellman pentru autentificare

  n, g – numere mari n prim si (n-1)/2 prim

  g intreg cu proprietatea:

  orice p intre 1 si n-1 inclusiv poate fi scris ca $p = g^k \bmod n$.

  x nu poate fi calculat din $g^x \bmod n$



Secure RPC - caracteristici

Fiecare principal (utilizator, calculator) are o pereche de chei (pastrate intr-o BD impreuna cu numele utilizatorului):

- Publica, pastrata in clar
- Secreta, pastrata in forma criptata DES cu password-ul principal-ului.

Un principal:

- Isi probeaza identitatea aratand ca poate decripta cheia secreta criptata
- Combina cheia secreta cu cheia publica a partenerului, ambii ajungand independent la o cheie de sesiune (SK) comuna, cunoscuta
- Aceasta cheie este folosita pentru a stabili o cheie de conversatie (CK).

Secure RPC – Local Login

U → C: U, PW

C: 1. Regaseste din baza de date publica de chei o inregistrare user continand: username, cheia publica user, {cheia secreta user}$_{PW}$

2. Decripteaza cheia secreta folosind PW si memoreaza cheia secreta in procesul keyserver

Unde:

U user

C client

PW password

Protocol folosit ptr autentificare client

1. C: 1. Primeste cheia publica server de la baza de date publica de chei
   2. Genereaza session key $(SK)_{c,s}$ pentru utilizare intre client si server
2. C → S: trimite C, $\{CK\}_{SK(c,s)}$, $\{window\}_{CK}$, $\{t_1, window+1\}_{CK}$
3. S: 1. Primeste cheia publica a clientului de la BD publica de chei
   2. Genereaza cheia de sesiune $(SK)_{c,s}$ folosita intre client si server
   3. Decripteaza cheia de conversatie CK folosind SK
   4. Decripteaza t_1, window si window+1
   5. Memoreaza intr-o tabela de credentiale, cu indexul ID: C, CK, window, t_1
4. S → C: $\{t_1 - 1\}_{CK}$, ID
5. C: Memoreaza ID si CK in procesul key server

Unde:	
C / S	client / server
CK	cheia de conversatie
window	timpul de viata al cheii CK

SK	cheia secreta generata de client si server
t_1	amprenta de timp originala

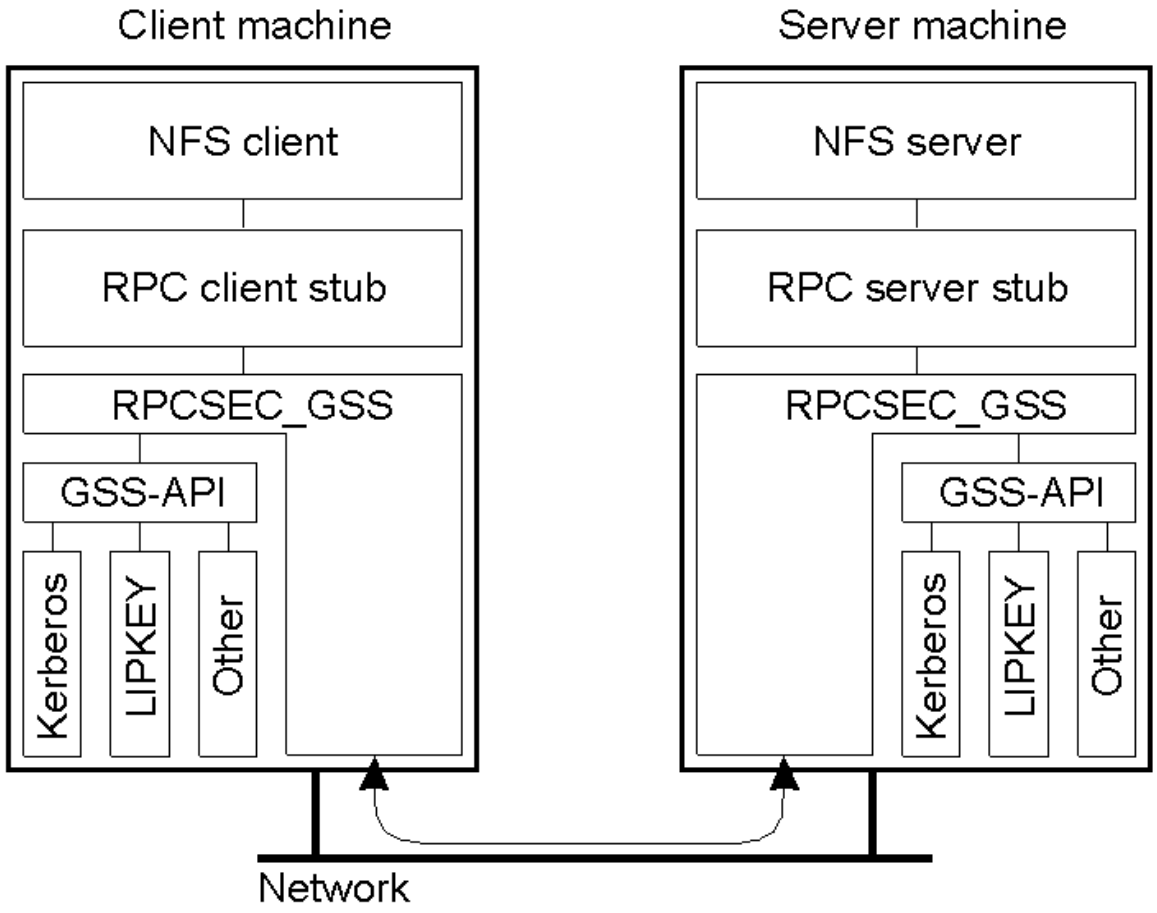
Tratare cereri dupa autentificare

6. C → S: ID, $\{t_n\}_{CK}$
7. S → C: $\{t_n - 1\}_{CK}$, ID

Unde:

- C client
- S server
- CK cheia de conversatie
- ID index client
- t_n amprenta de timp curenta

Secure RPC in NFS versiunea 4



GSS – Generic Security Service

Incorporeaza cai standard de securitate (Kerberos, LIPKEY - Low Infrastructure Public Key)

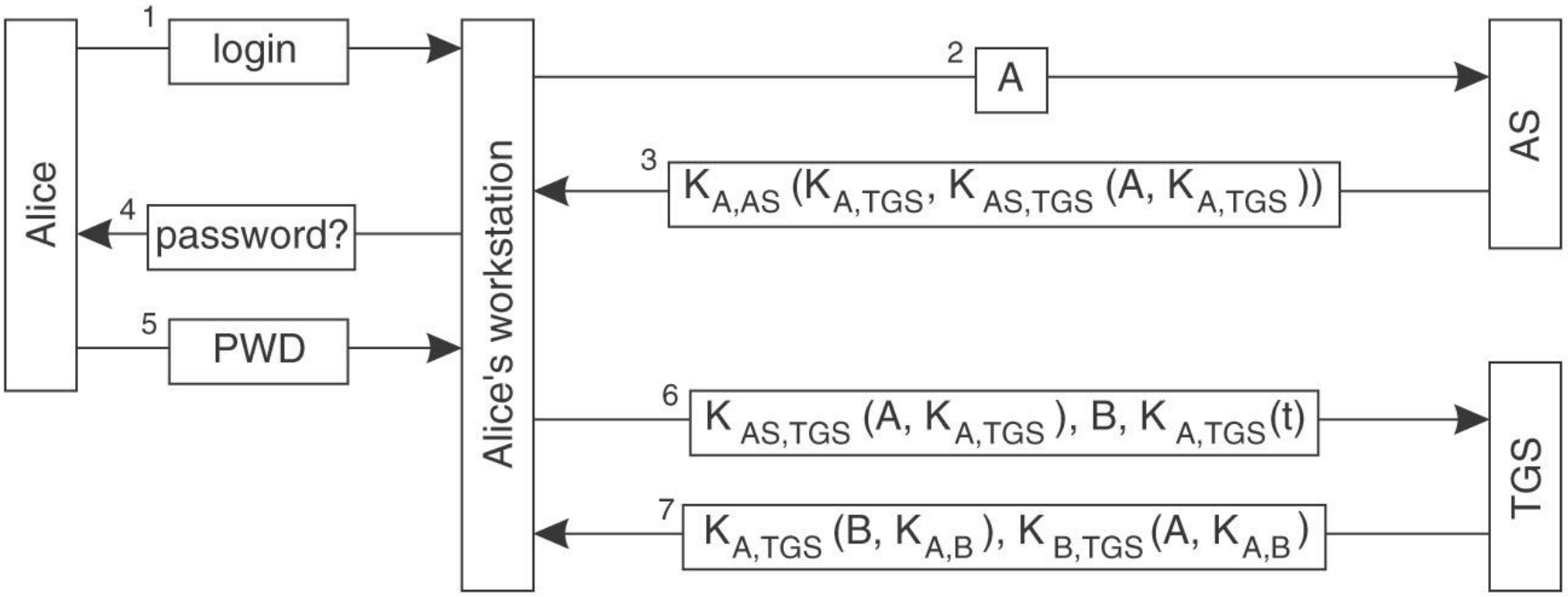
Kerberos

Protocol autentificare dezvoltat in proiectul Athena la MIT

Sistem de securitate care ofera:

- Autentificare
- Autorizare
- Confidentialitate mesaje

Autentificarea in Kerberos



AS – Authentication Server

TGS – Ticket Granting Server

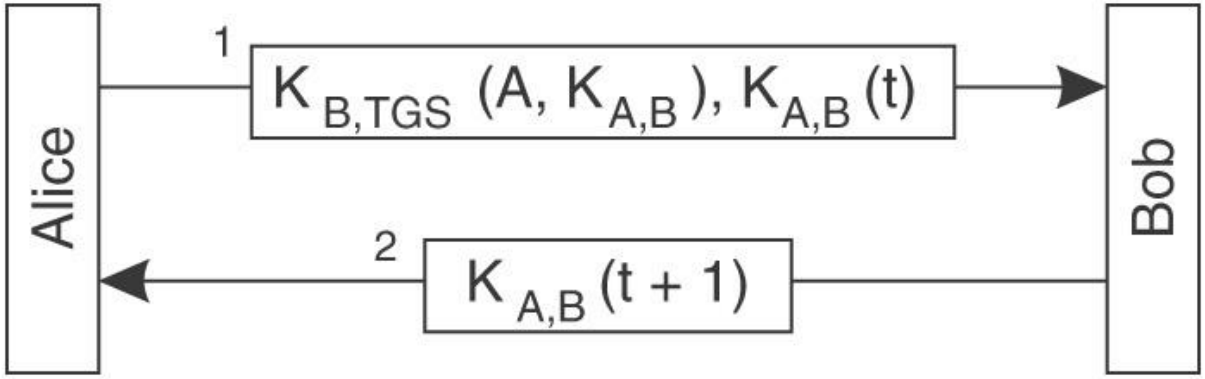
B – serviciu folosit de Alice

A – login name Alice

password folosit ptr decriptare mesaj 3

PWD – password

Invocarea serviciului



Securitatea sistemelor distribuite

Partea a 2-a

Controlul Accesului

Bazat pe 2 premise

- identificarea corecta a utilizatorului
  - facuta prin autentificare
  - nici un utilizator sa nu poata lua drepturile de acces ale altuia
- protejatarea informatiei despre drepturile de acces contra modificarilor neautorizate

Controlul Accesului – Elemente de baza

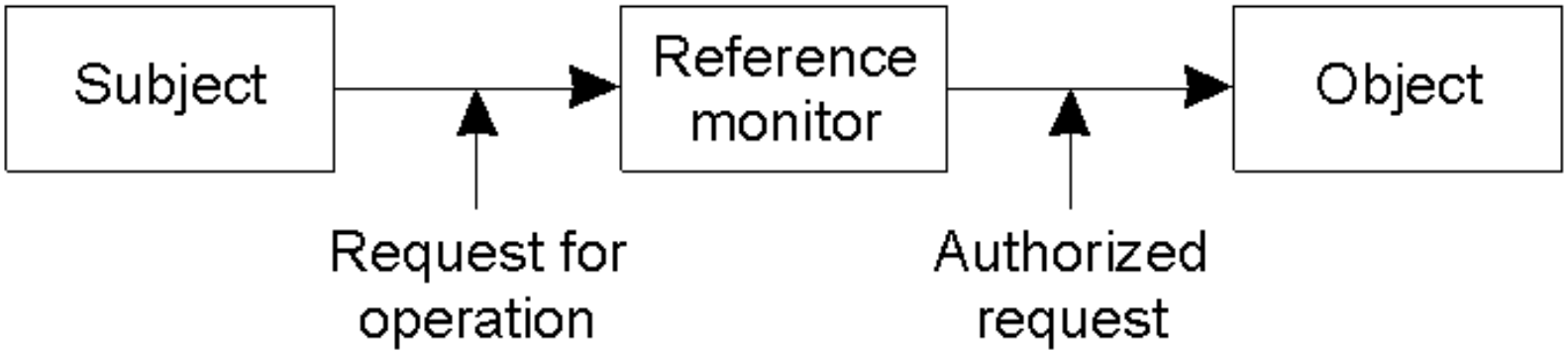
Problema: Are subiectul **s** dreptul de access **a** la obiectul **o**?

- Tuplul (s, o, a) constituie Autorizatia

Solutia: Controlul accesului este o functie f (s, o, a) care intoarce *true* sau *false*

Reference monitor implementeaza aceasta functie

- toate cererile senzitive trec prin Reference monitor
- monitorul decide daca operatia poate continua



Clasificarea modelelor de acces

Model de securitate

- o reprezentare mai precisa si mai detaliata a unei politici de securitate
- folosit ca referinta pentru construirea securitatii si pentru evaluarea ei

Clasificare dupa posibilitatea de transfer al drepturilor

- Discretionare - Discretionary Access Control (DAC)
  - utilizatorii pot transfera altora drepturile pe care le detin
- Mandatorii - Mandatory Access Control (MAC)
  - utilizatorii nu pot transfera drepturile pe care le detin

Clasificare dupa obiectul controlat

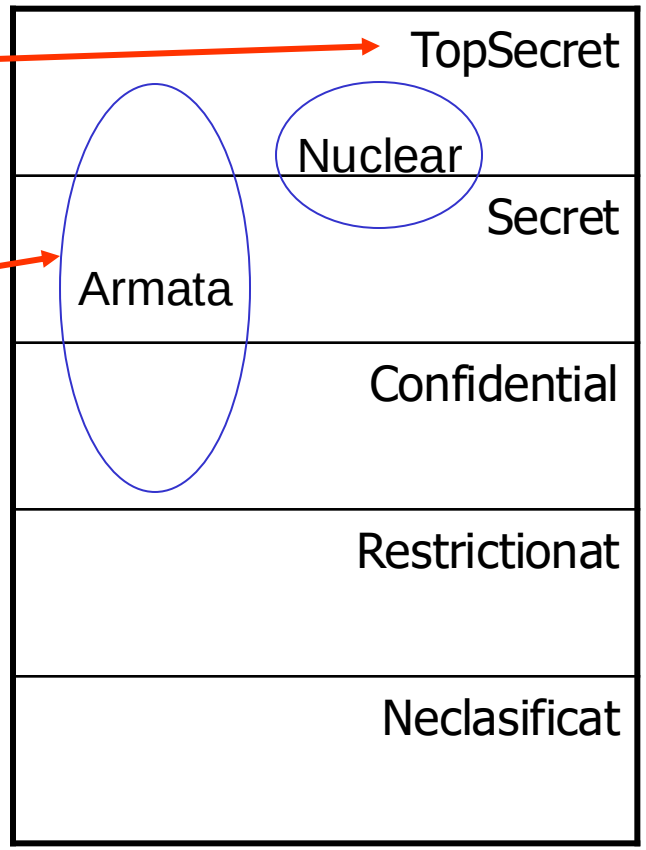
- fluxul informatiilor
  - modele multinivel
- accesul la informatii
  - matrice de acces,
  - RBAC – Role Based Access Control

Model de confidentialitate multinivel

Fiecare obiect

- are un nivel de senzitivitate sau **rang**: top secret, secret, ...
- este asociat cu unul sau mai multe proiecte (**compartimente**)
  - accesul este permis celor care "trebuie sa stie" pentru ca lucreaza in proiectul respectiv

Clasa de acces a unui obiect este combinatia <rang, compartimente>



Exemple

C1 = (TopSecret, {Nuclear, Armata})

C2 = (TopSecret, {Nuclear})

C3 = (Confidential, {Armata})

Fiecare subiect
- are o **autorizare** (clearance) exprimata ca o clasa de acces <rang, compartimente>
  - la ce nivel de senzitivitate are acces
  - in ce proiecte lucreaza

Accesul unui subiect la un obiect se bazeaza pe relatia de **dominanță** intre clase de acces
- Ci domina Ck (sau Ck este dominat de Ci), Ci $\geq$ Ck $\Leftrightarrow$
  rang (Ci) $\geq$ rang (Ck) si
  compartimente (Ci) $\supseteq$ compartimente (Ck)
- Ci domina strict Ck, Ci > Ck $\Leftrightarrow$
  rang (Ci) > rang (Ck) si
  compartimente (Ci) $\supset$ compartimente (Ck)
- Ci si Ck sunt incomparabile Ci <> Ck daca
  nici Ci $\geq$ Ck nici Ck $\geq$ Ci

Exemple

Clase de acces

C1 = (TopSecret, {Nuclear, Armata})

C2 = (TopSecret, {Nuclear})

C3 = (Confidential, {Armata})

C1 $\geq$ C2

C1 > C3

TopSecret > Confidential si

{Nuclear, Armata} $\supset$ {Armata}

C2 < > C3

Controlul accesului: Exemplu

Rang	Subiect	Obiect
Top secret	Ion, Rodica	Fisiere de personal
Secret	Sanda, Vasile	Fisiere e-mail
Confidential	Costi, Ioana	Fisiere log
Neclasificat	Mara, Mihai	Fisiere numere telefon

Senzitivitatea scade de sus in jos

Un singur compartiment

Cerinta 1 - Subiectii au acces doar la informatia pentru care au clasa de acces necesara

- Ioana nu are acces la Fisiere de personal
- Rodica are acces la toate fisierele

Cerinta 2 – Informatia sa nu se scurga spre clase de acces inferioare

- Rodica **poate citi** Fisiere de personal dar **nu** **trebuie sa le poata scrie** in Fisiere log pentru accesul Ioanei

Modelul Bell-La Padulla (BLP)

Pastreaza secretul - previne divulgarea neautorizata a informatiei; ex. secrete militare

Securitatea simpla (*no-read-up*)

- Subiectele au acces doar la informatia pentru care au clasa de acces necesara
- Formal: Un subiect s are acces *read* la un obiect o doar cand clasa subiectului domina clasa obiectului

$$C(s) \geq C(o)$$

Proprietatea Star (*) (*no-write-down*)

- Previne fluxul de informatie inspre obiecte cu clase de acces inferioare sau incomparabile
- Formal: Un subiect s care are acces *read* la un obiect p, poate avea acces *write* la un obiect o doar cand clasa de acces a lui o domina clasa lui p

$$C(o) \geq C(p)$$

Modelul de integritate Biba

Defineste nivele de integritate (I) analoage nivelelor de senzitivitate. Regulile sunt insa diferite.

Integritatea simpla

- Subiectul s poate modifica (acces *write*) un obiect o doar daca o are integritatea mai mica

$$I(s) \geq I(o)$$

- Justificare: un subiect s cu integritate mai mica ar scadea integritatea obiectului o modificat de el

Proprietatea-*

- Daca subiectul s are acces *read* la obiectul p cu integritatea I(p), atunci el poate avea acces *write* la obiectul o doar daca

$$I(p) \geq I(o)$$

- Justificare: un obiect p cu integritatea mai mica ar scadea integritatea obiectului o

Caracteristici comune

Modelele Bell La-Padula si Biba

- nu specifica modul de definire sau de modificare a claselor de acces si de autorizare
- nu trateaza delegarea sau transferul drepturilor de acces

Topicile sunt tratate de alte modele

- ideea generala (Graham-Denning):
  - definirea unei **matrice de acces** care sa specifice drepturile de acces pentru fiecare combinatie de subiecte si obiecte

Exemplu matrice de acces

Drepturile proceselor P1 si P2 asupra

fisierelor f1 si f2

proceselor P1 si P2

	f1	f2	P1	P2
P1	read write own	read	read write execute own	write
P2	append	read own	read	read write execute own

Modelul Graham - Denning

Modelul este definit in termeni de stari si tranzitii

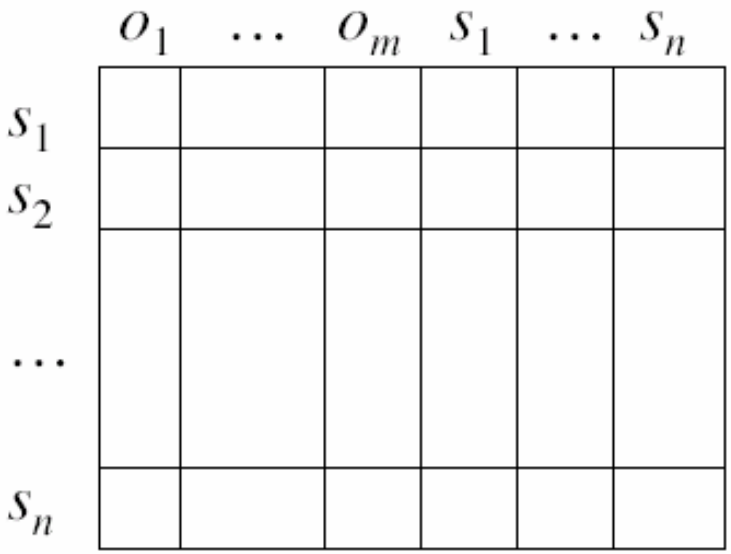
- o stare este reprezentata de o matrice de drepturi
- tranzitiile intre stari sunt descrise prin actiuni executate de subiecti

subiecti	S = {s1,...,sn}	
obiecte	O = {o1,...,om}	
drepturi	R = {r1,...,rk}	
intrari	A [s,o] $\subseteq$ R	drepturile din {r1,...,rk} pe care subiectul s le are asupre obiectului o

Graham-Denning

drepturile sunt definite ca

- actiuni A [s,o] pe care subiectul s le poate executa asupra obiectului o
- actiuni A[si,sj] pe care subiectul si le poate executa asupra subiectului sj



Proprietari si controlori

- fiecare **obiect** are un subiect **propietar** (owner) cu drepturi speciale
- fiecare **subiect** are un alt subiect cu drepturi speciale (**controlor**)

Modelul propune un set fix de actiuni primitive executate de subiectul x;

In urmatorul tabel:

- **r** reprezinta un drept
- **r*** inseamna ca dreptul **r** transmis de **x** lui **s** este *transferabil*, adica **s** poate transfera r sau r* altor subiecte

Actiuni primitive

Actiune	Preconditie	Efect
create object o	-	Adauga coloana o la A; adauga *Owner* la A[x,o]
delete object o	*Owner* in A[x,o]	Sterge coloana o
create subject s	-	Adauga linia s si col s la A; adauga *Control* la A[x,s]
delete subject s	*Control* in A[x,s]	Sterge linia s si col s
read access rights of s on o	*Control* in A[x,s] sau *Owner* in A[x,o]	Citeste A[s,o]
grant access right r to s on o	*Owner* in A[x,o]	Adauga r la A[s,o]
delete access right r of s on o	*Control* in A[x,s] sau *Owner* in A[x,o]	Sterge r din A[s,o]
transfer right r or r to s on o*	r* in A[x,o]	Adauga r sau r* la A[s,o]

Model Harrison-Ruzzo-Ullman – HRU

Harrison-Ruzzo-Ullman (HRU) bazat pe:

- S set de subiecte
- O set de obiecte
- R set de drepturi de acces
- O matrice de acces $M = (M_{so})_{s \in S,\, o \in O}$
- intrarea M_{so} este un subset din R specificand drepturile subiectului s asupra obiectului o sau asupra unui alt subiect s'

Operatii primitive simple din care se pot construi comenzi

- **enter** r into M_{so}
- **delete** r from M_{so}
- **create subject** s'
- **delete subject** s'
- **create object** o
- **delete object** o

Controlul drepturilor de acces se face la nivelul comenzilor

Comenzi

Comenzile sunt combinatii de actiuni primitive; au formatul:

command $c\,(x_1,.....,x_k)$
if r_1 in $M_{s1,o1}$ **and**
if r_2 in $M_{s2,o2}$ **and**
⋮
if r_m in $M_{sm,om}$
then $op_1,.....,op_n$
end

$s_1,.....,s_m$ si $o_1,.....,o_m$ sunt subiecti si obiecte care apar in lista de parametri $x_1,.....,x_k$

Daca toate conditiile sunt indeplinite atunci se executa lista de operatii

Sunt acceptate si comenzi mono-operatie

Exemple

Crearea unui fisier: s creaza o si devine proprietar

command CREATE_FILE(s,o)
create object o
enter own into *M*[s,o]
end CREATE_FILE

Transferul unui drept; de ex "read": s1 da un drept lui s2

command CONFER_READ(s1,o,s2)
if own $\in$ *M*[s1 ,o]
then enter read into *M*[s2 ,o]
end CONFER_READ

Revocarea unui drept; de ex "write":

command REVOKE_WRITE(s1,o,s2)
if (own $\in$ *M*[s1 ,o]) and (write $\in$ *M*[s2 ,o])
then delete write from *M*[s2 ,o]
end REVOKE_WRITE

Ce rezolva HRU - Sisteme de Protectie

Un **sistem de protectie**:

- Set finit de subiecti, obiecte, drepturi si comenzi

Un sistem de protectie este unul stari-tranzitii

- Fiecare stare este reprezentata de o instanta a matricei de acces
- comenzile fac tranzitia de la o stare la alta

Definitie *scurgere drepturi*. O **stare** M lasa sa se ***scurga*** dreptul ***r*** daca exista o comanda ***c*** care adauga dreptul ***r*** intr-o intrare din *M* care anterior nu continea ***r***. Mai precis:

- exista ***s*** si ***o*** astfel ca $r \notin M_{so}$ si
- dupa executia comenzii ***c***, $r \in M'_{so}$

Definitie *stare sigura*. O **stare** *M* intr-un sistem de protectie este **sigura** relativ la un **drept *r*** daca nicio secventa de comenzi nu poate transforma *M* intr-o stare in care se scurge ***r***.

Nota: Faptul ca un drept se poate "scurge" nu este neaparat rau; multe sisteme permit subiectilor sa delege drepturi de acces altor subiecti

Exemplu de sistem "nesigur"

Fie comenzile:

command grant_execute (*s*,*p*,*f*)
  if own in $M_{s,f}$
  then enter execute into $M_{p,f}$
end

command modify_own_right (*s*,*f*)
  if execute in $M_{s,f}$
  then enter write into $M_{s,f}$
end

Exemplu

Bob a dezvoltat o aplicatie P1 si vrea ca ea sa fie executata de alt utilizator (Tom) dar nu modificata de acesta

Sistemul anterior este nesigur; permite urmatoarea secventa:

- Bob: grant_execute (Bob, Tom, P1)
- Tom: modify_own_right (Tom, P1)

face ca in M, intrarea $M_{Tom,P1}$ sa contina dreptul de acces write

Siguranta in modelul HRU

Problema sigurantei poate fi formulata astfel:

Este decidabil daca un *subiect* ar putea obtine vreodata un anumit *drept* relativ la un *obiect* ?

Teorema. Data fiind matricea de acces *M* si dreptul *r*, verificarea sigurantei lui *M* relativa la *r* este o problema nedecidabila in cazul general.

Problema sigurantei

- este decidabila pentru sisteme de protectie mono-operatie
- nu este intotdeauna decidabila pentru alte tipuri de sisteme de protectie
  - protectia in UNIX cere mai mult de o operatie per comanda

Siguranta in modelul Take-Grant

Modelul are patru operatii primitive

- reprezentate prin grafuri
  - subiecti si obiecte -> noduri
  - drepturi -> arce etichetate, orientate de la subiect la obiect

Urmatoarele operatii sunt executate de subiectul **s**

create(o, r):



revoke(o,r):



Delegare drepturi

grant(o,p,r):



Subiectul s “deleaga” lui o dreptul de acces r asupra lui p.

Preconditii:

- s are drept de delegare (Grant) a unor drepturi catre o
- s are dreptul r asupra lui p

Preluare drepturi

take(o,p,r):



Subiectul **s** preia de la **o** dreptul de acces **r** asupra lui **p**

Preconditii

- **s** are drept de preluare (Take) de drepturi de la **o**
- **o** are dreptul **r** asupra lui **p**

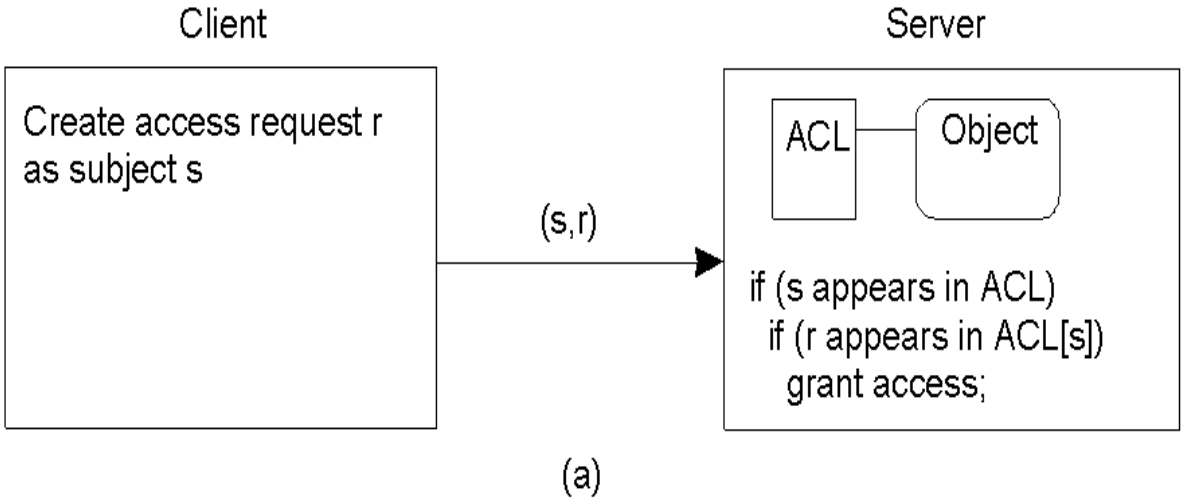
Cu acest model se poate decide daca

- un subiect poate partaja un obiect cu un alt subiect
- un subiect poate "fura" accesul la un obiect de la un alt subiect

Implementare Matrice Control Acces (MCA)

ACL (Access Control List)

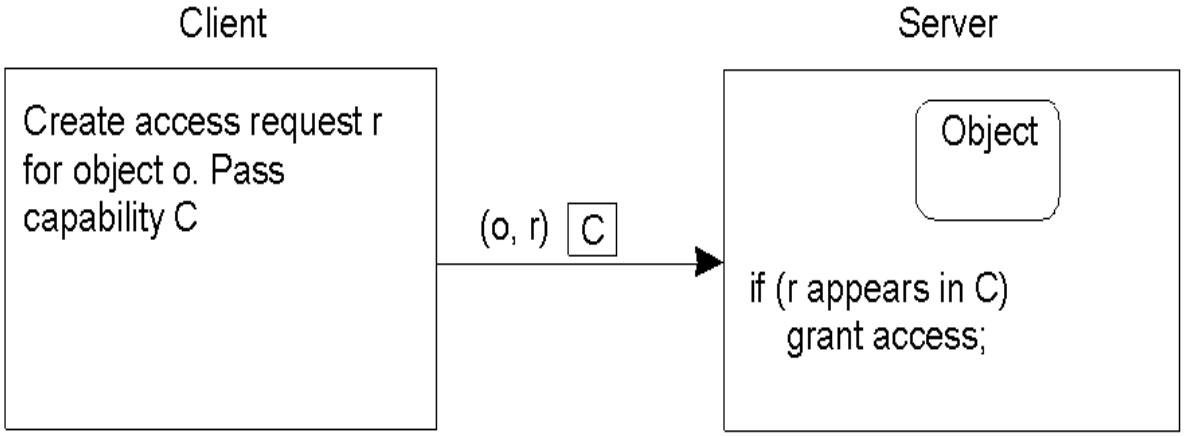
fiecare obiect O pastreaza MCA[*,O]



(a)

Capabilitati

fiecare subiect S are capabilitatile din MCA[S,*]



(b)

Probleme cu modelele "clasice"

Matrice acces (respectiv ACL, C-list):

Nu pot reprezenta modele de acces mai complexe

- de ex. bazate pe reguli cum ar fi competenta,
- cele mai reduse privilegii sau
- conflict de interese

Nu suporta schimbari dinamice

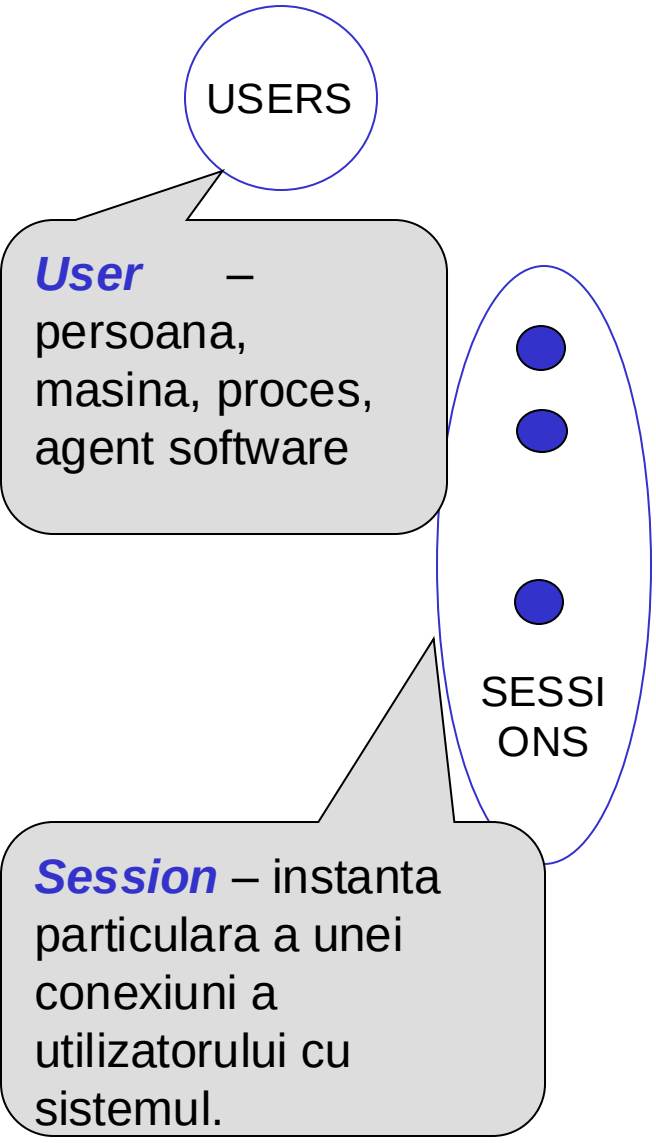
- modificarea drepturilor unui subiect necesita inspectarea ACL a fiecarui obiect
- determinarea subiectilor care au acces la un obiect necesita inspectarea tuturor listelor de Capabilitati

Nu pot gestiona drepturi de acces determinate de continutul, atributele obiectelor sau de context

Nu inregistreaza utilizarea permisiunilor

- Un subiect poate utiliza permisiunile ori de cate ori

RBAC - Role Based Access Control



User – persoana, masina, proces, agent software

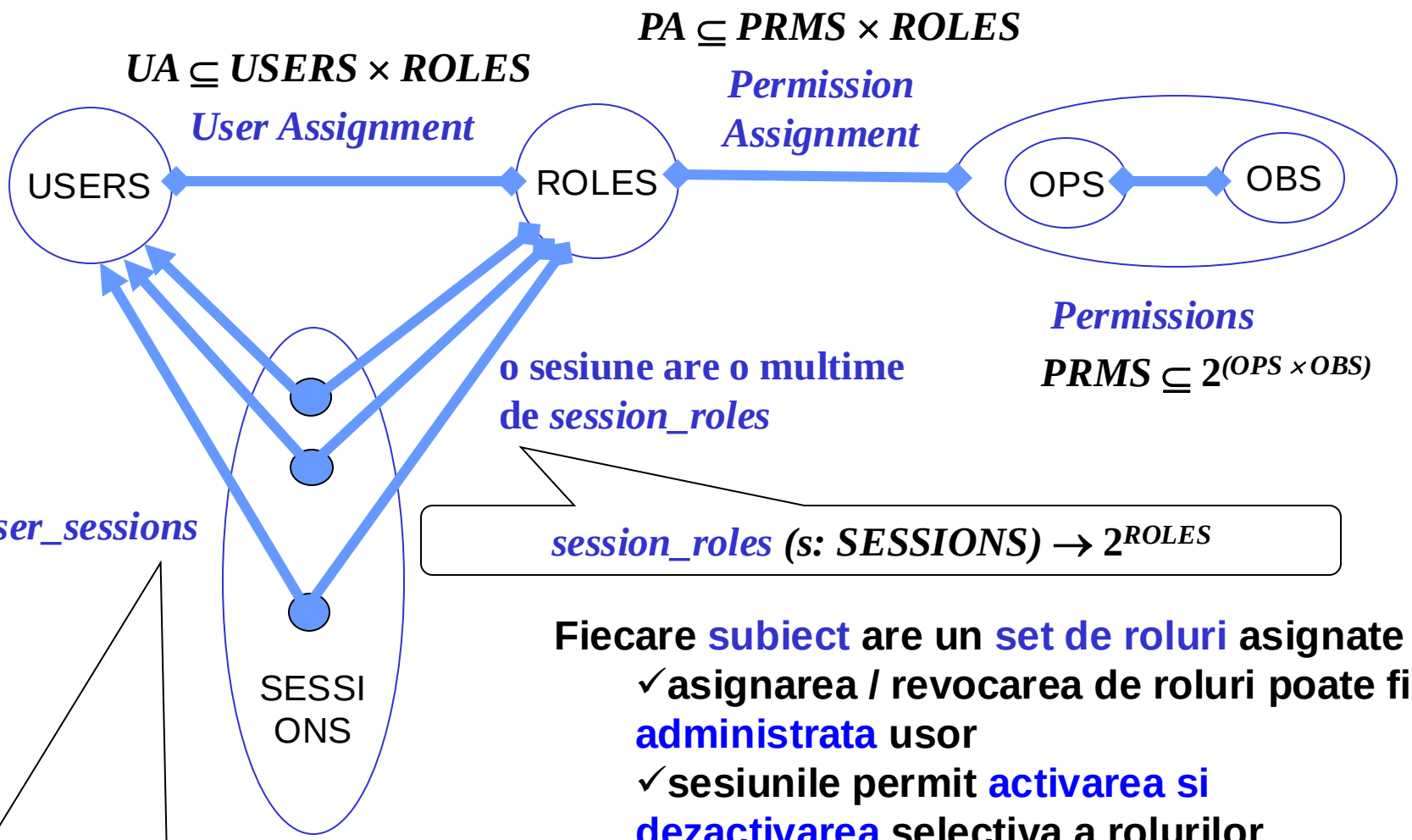
Role – functie in contextul unei organizatii (pozitie, responsabilitati)

Permission – un mod de acces (operations OPS) asupra obiectelor (objects OBS)

Session – instanta particulara a unei conexiuni a utilizatorului cu sistemul.

Modelul de referinta pentru Core RBAC ($RBAC_0$)
defineste multimi de **Elemente de baza**

RBAC – relatii intre elementele de baza



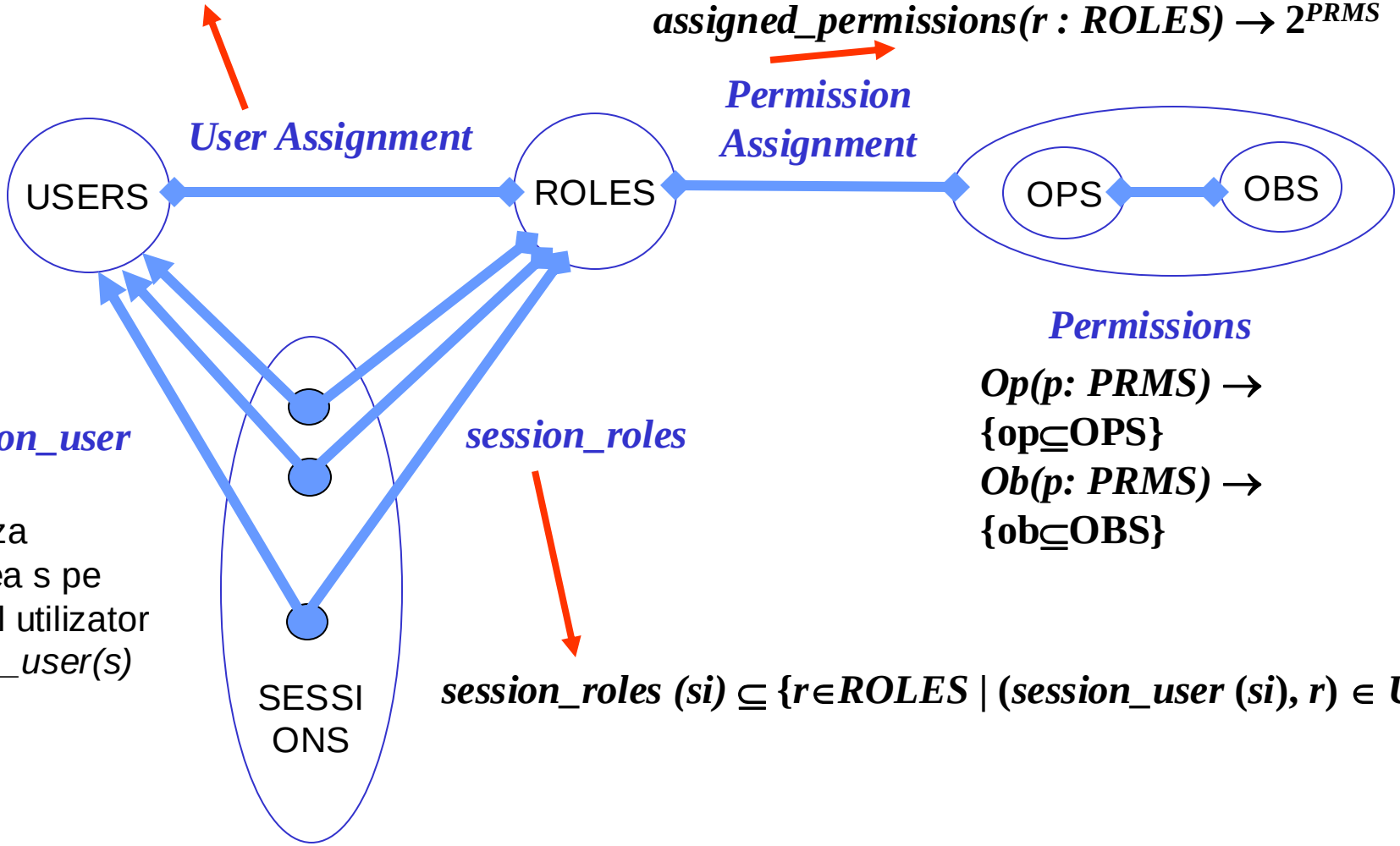
Fiecare subiect are un set de roluri asignate

- ✓**asignarea / revocarea de roluri poate fi administrata usor**
- ✓**sesiunile permit activarea si dezactivarea selectiva a rolurilor**
- ✓**un utilizator poate avea simultan, intr-o sesiune, permisiuni din mai multe roluri**
- ✓**modelul suporta astfel principiul privilegiilor minime (least privilege)**

RBAC – functii de mapare

$assigned_users(r{:}ROLES) = \{u \in USERS \mid (u, r) \in UA\}$

$assigned_permissions(r : ROLES) \rightarrow 2^{PRMS}$



session_user

mapeaza sesiunea s pe singurul utilizator *session_user(s)*

session_roles

$session_roles\ (si) \subseteq \{r \in ROLES \mid (session_user\ (si), r) \in UA\}$

Permissions

$Op(p{:}\ PRMS) \rightarrow \{op \subseteq OPS\}$

$Ob(p{:}\ PRMS) \rightarrow \{ob \subseteq OBS\}$

Rolurile (si nu subiectii) au asociate permisiuni

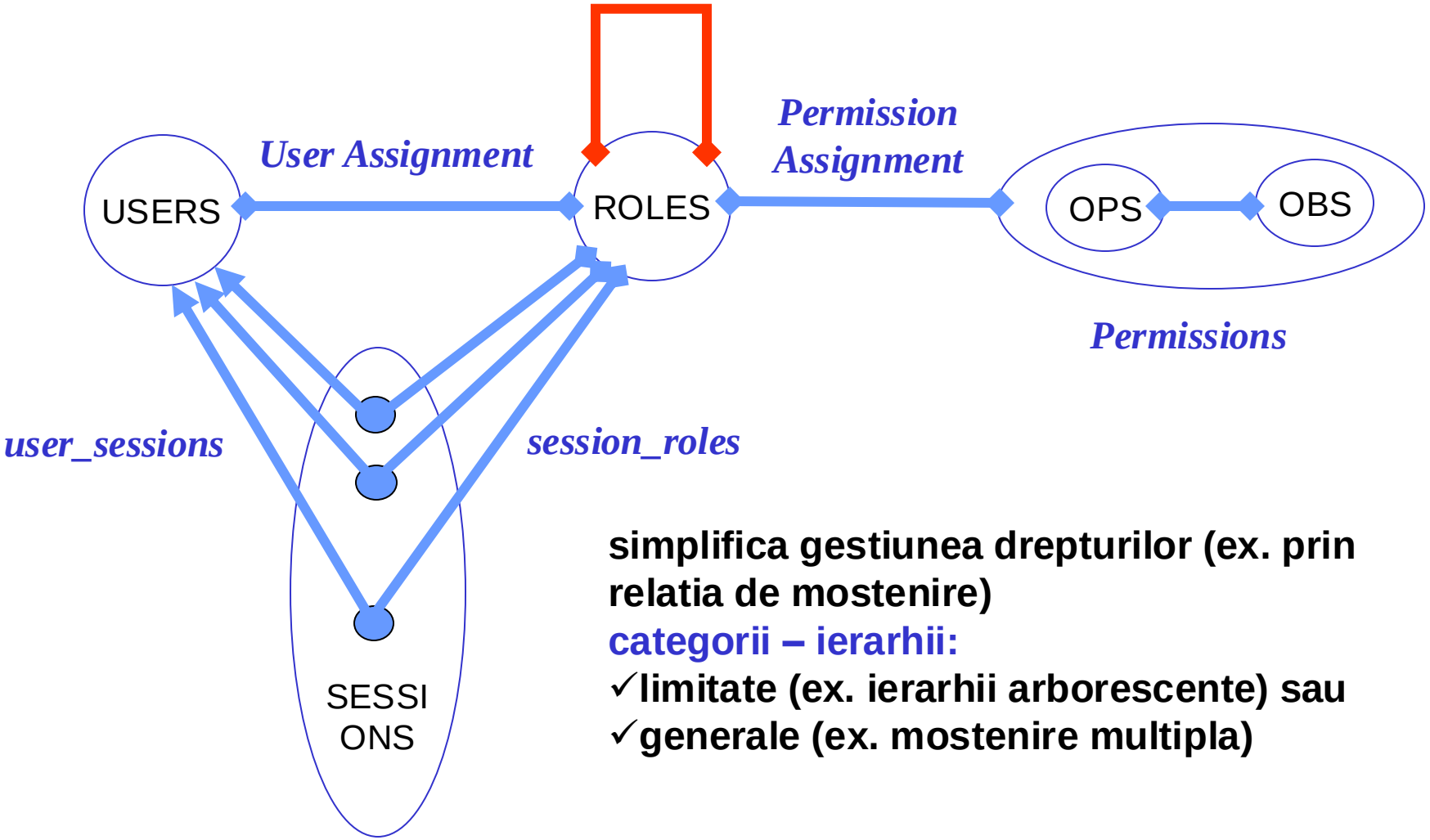
- ✓**permit predefinire roluri**
- ✓**relatia rol – permisiuni se schimba mai rar ➔ administrare mai usoara**

RBAC ierarhic

relatie intre roluri: rolul r1 include rolul r2, notat r1 ≥ r2 ⇔

RolePermissions (r2) ⊇ RolePermissions (r1) – permisiile lui r2 sunt si ale lui r1

si AssignedUsers (r1) ⊇ AssignedUsers (r2) – user-ii lui r1 sunt si ai lui r2



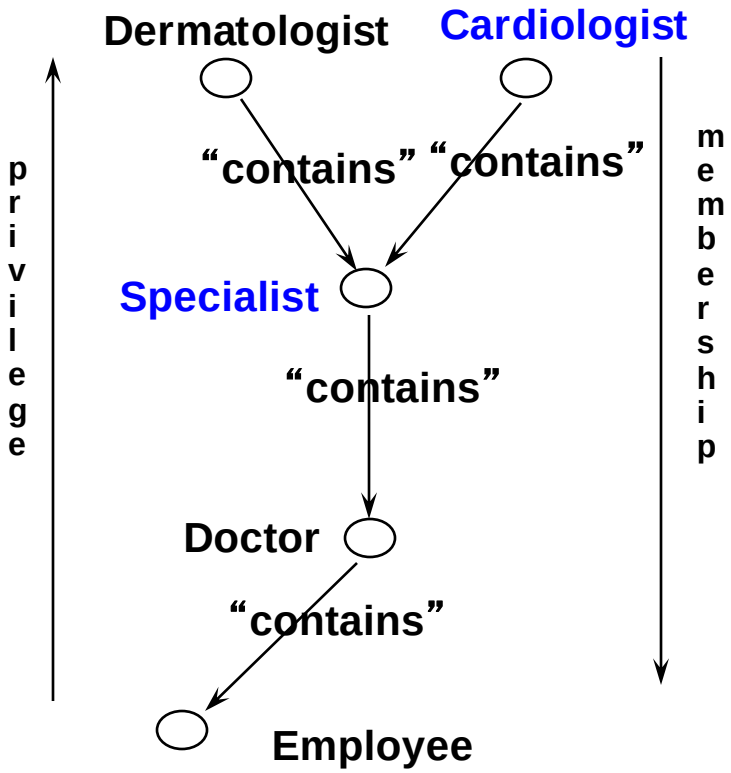
simplifica gestiunea drepturilor (ex. prin relatia de mostenire)

categorii – ierarhii:

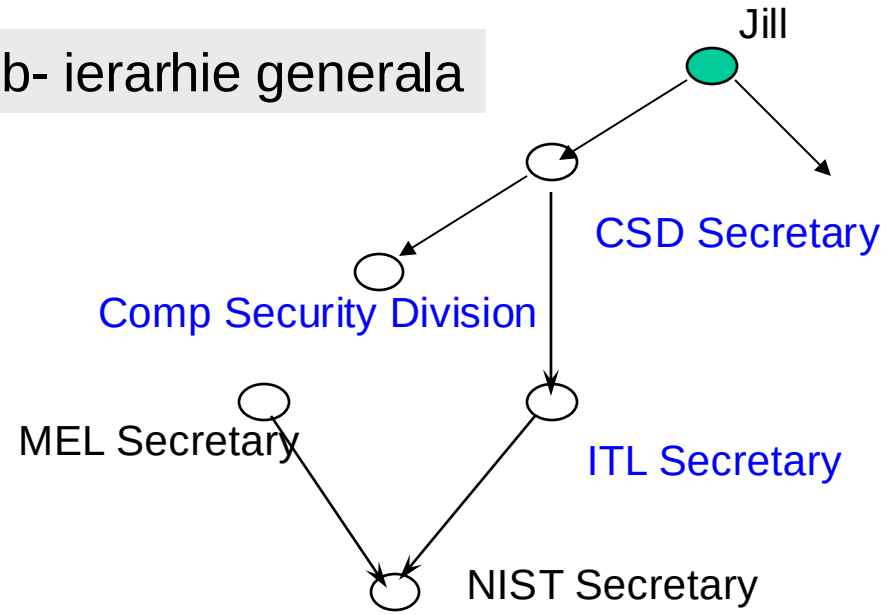
- ✓**limitate (ex. ierarhii arborescente) sau**
- ✓**generale (ex. mostenire multipla)**

Ierarhia de roluri: exemple

a-ierarhie limitata



b- ierarhie generala



Rolul CSD Secretary poate fi definit din privilegiile Comp Security Division si ITL Secretary care au roluri subordonate

Un Cardiolog mosteneste permisiunile unui Specialist la care adauga altele

RBAC Separarea sarcinilor

Evita conflictul de interese

1. Forma generala

Constrangere de cardinalitate pe un set de roluri

S = (set roluri, n) – niciun utilizator nu poate avea **n** sau mai multe roluri din set

2. Excludere mutuala a permisiunilor

roluri incompatibile: nu pot fi asumate simultan de un utilizator

daca ri, rk incompatibile =>

ri $\in$ AssignedRoles(u) => rk $\notin$ AssignedRoles(u)

Doua forme

- ✓ **Static Separation of Duty Relations** – ex. rolurile ri si rk sa nu fie **asignate** ambele unui utilizator
- ✓ **Dynamic Separation of Duty Relations** - ex. rolurile ri si rk sa nu fie **active** in acelasi timp pentru un utilizator

Specificatie Functionala RBAC

Functii Administrative

Creare si Intretinere seturi de elemente

Creare si Intretinere Relatii

Functii Suport Sistem - Management sesiune

- CreateSession - Creaza User Session si da utilizatorului un set de roluri active implicite
- AddActiveRole – Adauga un rol ca rol activ in sesiunea curenta
- DropActiveRole – Elimina un rol activ din sesiunea curenta

Decizii de control al accesului

- CheckAccess – Determina daca subiectul are permisiunea sa execute o anumita operatie asupra obiectului.

Functii Administrative de informare (Review)

(M – Mandatory, O – Optional)

- AssignedUsers (M) - Intoarce setul de utilizatori asignat unui rol dat
- AssignedRoles (M) - Intoarce setul de roluri asignate unui utilizator dat
- RolePermissions (O) - Intoarce setul de permisiuni garantate unui rol dat
- UserPermissions (O) - Intoarce setul de permisiuni pe care un utilizator le are prin rolurile sale
- SessionRoles(O) - Intoarce setul de roluri active asociate cu o sesiune
- SessionPermissions (O) - Intoarce setul de permisiuni disponibile in sesiune (reuniunea tuturor permisiunilor asignate rolurilor active din sesiune)

Caracteristici RBAC

Calitati

- utilizatorul poate trece usor de la un rol la altul in cursul unei sesiuni, fara a schimba structura accesului
  - rezultat: RBAC mai scalabil ca matricile de acces
- permite tratarea conflictelor de interese (rolurile au reguli stricte)
- reduce overhead la administrare securitate la nivel de utilizator, obiect, permisiune

Relatia cu MAC si DAC

- RBAC este un model neutru fata de politicile de control al accesului
- poate coexista cu MAC sau DAC
  - accesul trebuie sa fie permis de RBAC si MAC / DAC

Utilizare

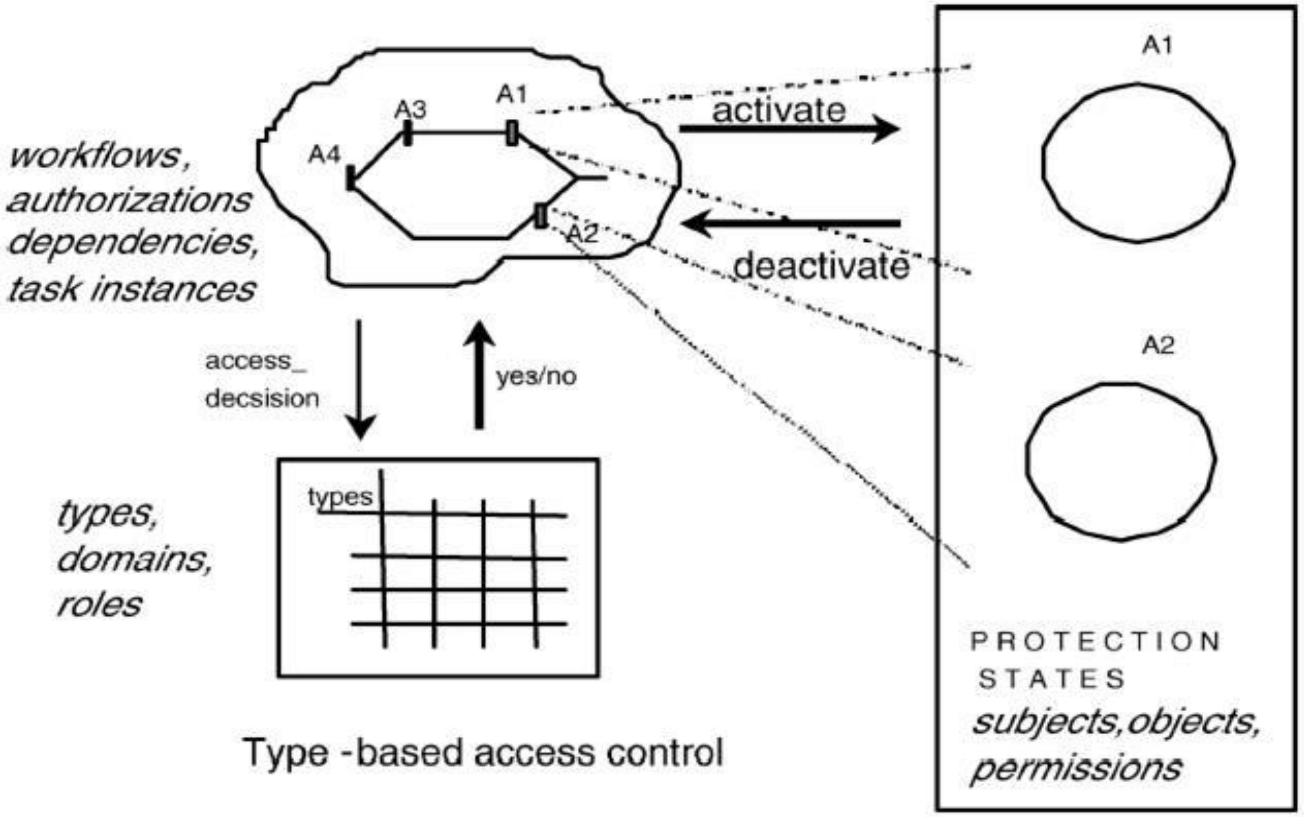
- Microsoft Active Directory, SELinux, FreeBSD, Solaris, Oracle DBMS, PostgreSQL 8.1, SAP R/3

TBAC – Task Based Access Control

Permisiunile sunt **activate** la inceputul fiecarui task si **dezactivate** la terminarea lui (model de **securitate activa**)

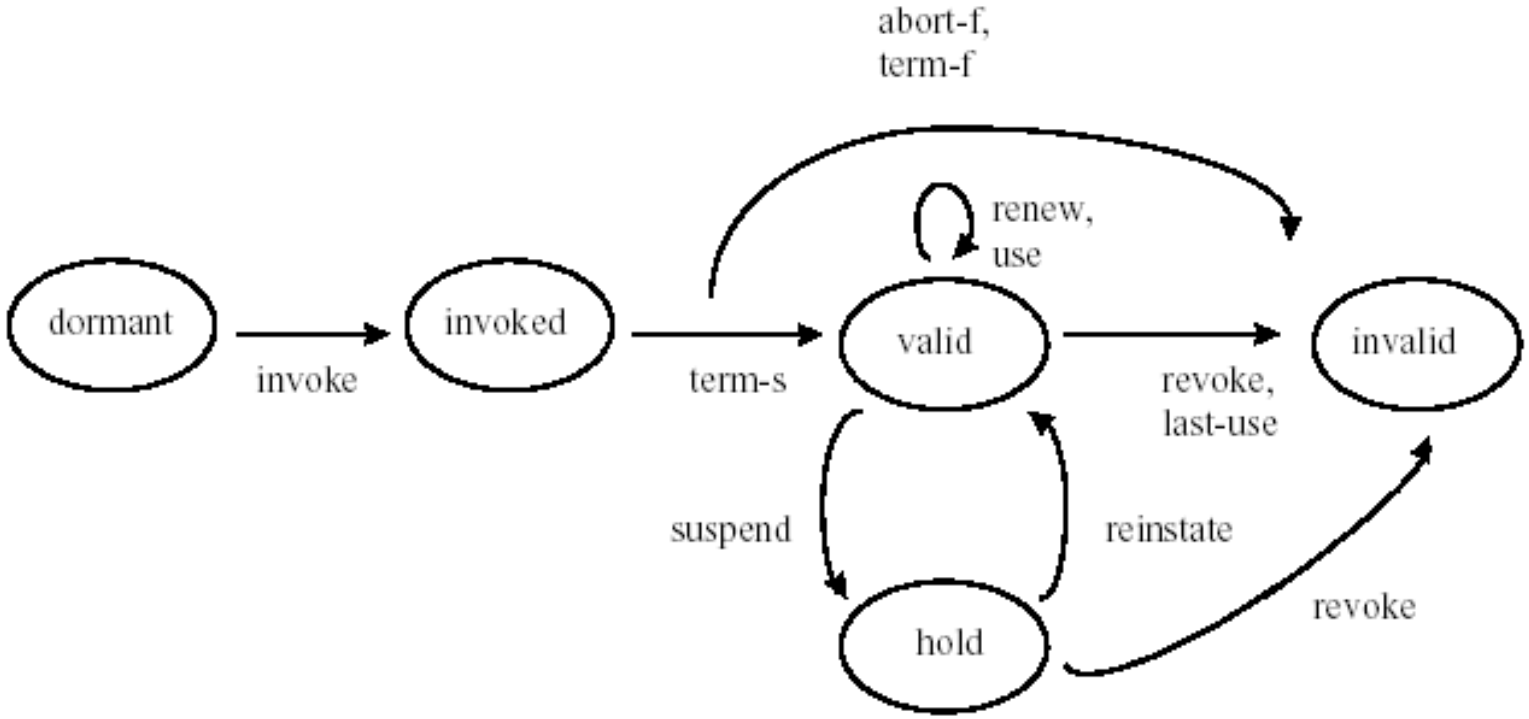
Setul de permisiuni (**Stare de protectie**) descrie drepturile taskului (subiect) asupra obiectelor

Combina controlul bazat pe **tipuri** (RBAC) si controlul bazat pe **instanta** si **utilizare**



Type -based access control

Procesarea autorizarii



Ciclu de viata cu mai multe stari:

este invocat un **pas de autorizare**

daca prelucrarea lui se termina cu succes devine valid, altfel invalid

valid – permisiunile sunt activate si pot fi "consumate"

pe parcurs, permisiunile pot fi suspendate temporar (hold)

la consumarea permisiunilor (sfarsitul ciclului de viata) pasul de autorizare devine invalid

Caracteristici TBAC

Admite **ciclul de viata** si pasii de procesare a autorizarilor

Introduce notiunea de **consum** asociata cu permisiunile (limitarea numarului de accese)

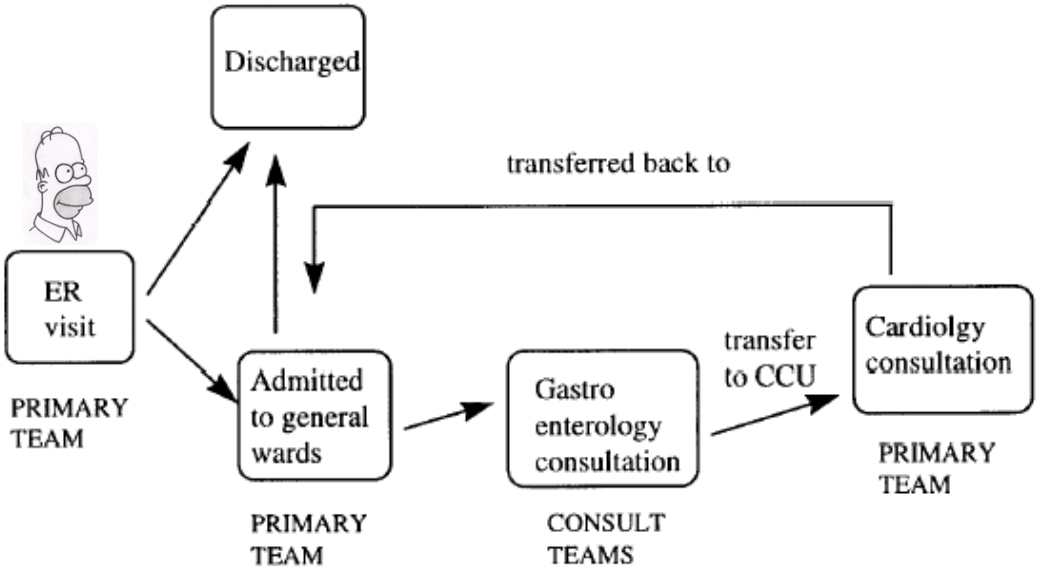
Formeaza baza **auto-administrarii** modelului de securitate (administrare cuplata cu activarea si terminarea taskurilor)

Limitari

- contextul este limitat la relatia cu pasii taskului
- dificultati in specificarea politicilor complexe, a delegarii si revocarii privilegiilor

TMAC - Scenariu

- pacientul vine la camera de garda
- este internat la medicina generala
- echipa decide sa faca o analiza la gastro-enterologie
- in timpul analizei are un atac de cord si este transferat la sectia de cardiologie
- dupa tratament revine la medicina generala
- pacientul se recupereaza si este eliberat din spital.



- personalul este organizat in **echipe** care ingrijesc bolnavul in diverse etape
- fiecare echipa este asociata unui **departament**
- membrii echipelor au **roluri** diferite
- echipele sunt **formate dinamic** (consultatie gastro)

Cerinte pentru controlul accesului

1. permisiunile asociate unui membru reflecta **competentele lui** - cardiologul poate prescrie medicatia pentru atac de cord
2. permisiunile se refera la un **anumit pacient** (obiect)
   - doar **membrii echipelor** de ingrijire a acestui pacient pot avea acces la fisele sale
   - recomandarea pentru gastro o da medicul din echipa acestui pacient
3. accesul este permis in **functie de context**
   - ex. **traseul pacientului** - echipa de generalisti are acces la fise doar cand pacientul a trecut la medicina generala
4. **activare colectiva** a permisiunilor la executie
   - cand pacientul trece la o alta unitate din spital, **noua echipa** are acces la fisa lui, nimeni altcineva
5. membrii pot **delega** permisiunile
   - medicul delega un rezident sa comande un medicament pentru pacient
6. permisiunile pot fi dezactivate
   - cand pacientul paraseste spitalul, permisiunile de acces la fisele sale **inceteaza**

Principalele idei TMAC

O echipa are:

un nume, t.

un set de membri / team users, TU.

un set de team roles, TR $\subseteq$ R (R = setul total de roluri)

un rol special team head (h), h € TR

un set de tipuri de obiecte, OT

un set de instante de obiecte, O

un set de team permissions TP, TP $\subseteq$ TR x OT

un context de colaborare cu doua componente

- user context (UC), UC : TR x TU
- object context (OC), OC : OT x O

Operational

- Primitive de grup
  - User-assign (user, team): asigneaza user la team.
  - User-deassign (user, team): inlatura user din team.
  - Team-activate (team): leaga team permissions la team users si la obiecte (TU si O).
  - Team-deactivate (team): dezactiveaza permisiunile pentru team.
- Primitive individuale
  - User-activate (user): activare permisiuni individual
  - User-deactivate (user): dezactivare permisiuni individual
- Contextul obiectelor se transmite de la o echipa la alta

TMAC – Concluzii

Centrat pe notiunea de echipa – colectie de utilizatori cu roluri diferite dar care au ca obiectiv comun indeplinirea unui task.

Permisiunile unui subiect depind de

- rolul sau
- grupul caruia ii apartine

Permisiunile sunt

- asignate unui rol (ca in RBAC) - model de securitate pasiva
- activate / dezactivate in functie de context - model de securitate activa

Context-Aware Access Control

Control acces in functie de conditiile de mediu

- timp: accesul este permis intr-o anumita perioada weekend, weekday
- loc: acces la un sistem securizat permis daca se face dintr-o incapere securizata fizic
- incarcarea unui sistem: rolul "high load" este activat la peste 70%

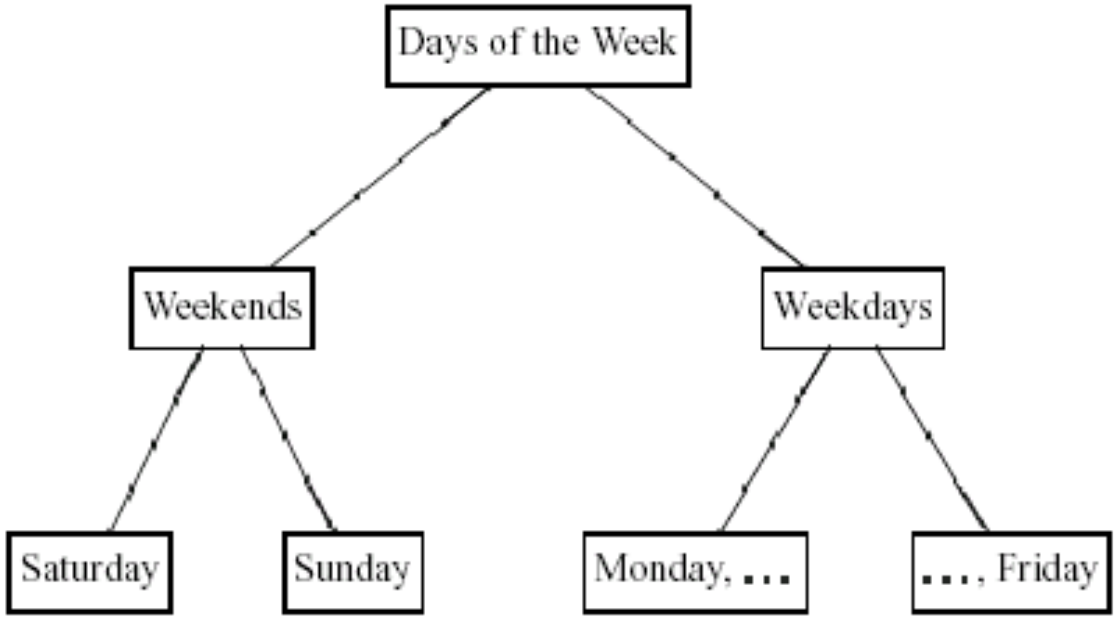
Extinde RBAC cu "environment roles" care surprind contextul

- rolurile sunt activate pe baza conditiilor de mediu
- revocarea se face cand conditiile nu mai corespund rolului

Rolurile de mediu au caracteristici comune cu cele clasice

- evitare conflicte (separarea intereselor)
  - uneori se doreste activarea unui singur rol la un moment dat
- ierarhia de roluri

unele Environment Roles pot fi activate pe durata unei sesiuni, dar schimbarea conditiilor reclama activarea altor roluri



Modelul

Din RBAC0:

se pastreaza Users, Roles, Permissions si Sessions.

adauga ER si EC,

ER - *Environment Roles*;

EC - *Environment Conditions* folosite in definirea rolurilor.

Relatii UserAssignment, PermissionAssignment

- User Assignment $UA \subseteq U \times R$
- Permission Assignment $PA \subseteq P \times R \times 2^{ER}$
  - PA asociaza o permisiune cu rolul unui subiect si
  - o conditioneaza de un set de environment roles ER active.

Functii care definesc ce roluri utilizator sau de mediu pot fi activate:

User: S -> 2^R

- Intr-o sesiune data S, pot fi activate anumite Roluri pentru un User

Request: EC -> 2ER

- Pe baza Environmental Conditions, un set de Environment Roles sunt activate la momentul cererii.

O cerere de permisiune *p poate fi* garantata daca:

(1) **p** este in setul de permisiuni asociate rolului r

***<p, r, e-set>* ϵ PA**

(2) Rolul **r** este in setul de roluri active ale utilizatorului care face cererea

(3) environment roles active in conditiile environmentale curente *EC* contin rolurile din ***e-set***.



Vă mulțumesc!